```
    "match": {
      "title": {
        "query": "Kava",
        "fuzziness": 1
      }
    }
  }
}
```

Setting fuzziness to 1 replaces one letter with all other combinations.

When searching for the text string "Java Complete Guide", we use a set of words to search for a book (or books) and usually expect the words to be treated individually (like a set of search words). However, we sometimes want to search for a phrase or sentence. That's when the `match_phrase` query is useful.

10.6 The match_phrase query

A matching phrase (`match_phrase`) query finds documents that exactly match a given phrase. The idea is to search for the phrase (group of words) in a given field in the given order. For example, if we are looking for the phrase "book for every Java programmer" in the synopsis of a book, documents are searched with those words in that order.

In our previous section (section 10.5) on `match` queries, we saw that words can be split individually and searched with an `AND/OR` operator when using a `match` query. The `match_phrase` query is the opposite. It returns results that match the search phrase exactly. The following listing illustrates the `match_phrase` query in action.

Listing 10.10 The `match_phrase` query in action

```
GET books/_search
{
  "query": {
    "match_phrase": {
      "synopsis": "book for every Java programmer"
    }
  }
}
```

Specifies a match_phrase query

Specifies the phrase (group of words) to be matched

The `match_phrase` query expects a phrase. It returns exactly one document because only one in our `books` index has that phrase in the `synopsis` field.

What if we drop a word or two in the search phrase? For example, suppose we remove *for* or *every* (or both) from the phrase "book for every Java programmer" and rerun the query. Unfortunately, the query won't return any results! This is because `match_phrase` expects the words in a search phrase to match the exact phrase, word by word. Searching for "book Java programmer" returns no results. But there is a fix to this problem: using the `slop` parameter.

The `slop` parameter allows us to ignore the number of words between the words in a phrase. We can drop the in-between words, but we must let the engine know how many words to drop. This is done by setting a value for the `slop` parameter. The `slop`